# Irodov Plus Classes - Full Project & Interview Guide

A Comprehensive Beginner-to-Advanced Codebase Breakdown, Architecture Overview, and Top Technical Interview Questions & Answers

---

## 1. Executive Project Overview

**Irodov Plus Classes** is a production-ready, full-stack educational portal designed for a premier physics coaching institute headed by Er. Jagannath Dutta (JD Sir). Built on the **MERN stack** (MongoDB, Express, React 19, Node.js), it solves two primary operational requirements: **(1) Student Onboarding & Admissions** through registration, login, and UPI receipt verification, and **(2) Digital Class Material Distribution** enabling the admin to upload lecture notes and problem sheets in PDF format, which students can search, preview inline in their browser, or download.

| Component | Technology | Key Role & Architectural Reason |
|---|---|---|
| Frontend Library | React 19 + Vite 7 | Modern reactive SPA with sub-second HMR and component modularity. |
| Styling System | Tailwind CSS v4 + DaisyUI | Utility-first styling with pre-built accessible components (modals, drawers). |
| Routing & Forms | React Router 7 + Hook Form | Client-side routing with high-performance uncontrolled form validations. |
| Backend Framework | Node.js + Express 5 | REST API gateway managing JSON parsing, streaming, and middleware pipelines. |
| Database & ODM | MongoDB Atlas + Mongoose 8 | Flexible NoSQL document store with strict schema definitions and validation. |
| Security & Auth | Bcrypt + JWT + Cookies | Adaptive 10-round salted password hashing and stateless session tokens. |
| File Storage | Multer (MemoryStorage) | Multipart stream parsing storing PDF buffers directly in MongoDB documents. |

## 2. Project Directory Tree & Codebase Blueprint

**Backend Architecture:**

• config/: Holds MongoDB connection logic (mongoose-connect.js) and environment-specific database credentials.
• Models/: Mongoose schemas enforcing database contracts: Student.js, Resource.js, ownermodel.js, Enroll.js, Image.js.
• Route/: Modular route controllers: studentRouter.js (auth & profile), resourceRouter.js (PDF uploads & streams), ownerRouter.js (admin auth), enroll.js.
• index.js: Main server entry point establishing CORS, parsing middleware, mounting routers, and launching the listener.

**Frontend Architecture:**

• src/utils/api.js: Dynamic API base URL resolver (seamlessly switches between localhost and production).
• src/components/Navbar.jsx: Stateful global navigation tracking student session via custom events.
• src/components/Login.jsx: Unified authentication modal supporting both Student (Login/Register) and Admin roles.
• src/components/Resources.jsx: Student repository with live search, topic/class filters, and PDF action buttons.
• src/components/OwnerResourceUpload.jsx: Admin dashboard for uploading and deleting PDF materials.
• src/components/ViewStudents.jsx: Admin view of all registered student accounts.
• src/App.jsx: Central route registry defining client endpoints (/resources, /owner, /enroll).

# 3. Deep-Dive Backend Code Walkthrough

## A. Student Model (Backend/Models/Student.js)

Defines the schema for student accounts. Crucially includes unique and lowercase flags on email to prevent duplicate registrations with varying capitalization, and enforces a minimum 6-character password constraint.

```
const studentSchema = new mongoose.Schema({
fullname: { type: String, required: true, trim: true },
email: { type: String, required: true, unique: true, lowercase: true, trim: true },
password: { type: String, required: true, minLength: 6 }, // Hashed bcrypt string
phone: { type: String, trim: true, default: '' },
classGrade: { type: String, trim: true, default: 'Class 12' },
createdAt: { type: Date, default: Date.now }
});
```

## B. Resource Model (Backend/Models/Resource.js)

Defines the structure of downloadable PDF materials. Stores binary data directly as a Buffer along with its MIME type (application/pdf), original filename, and size in bytes.

```
const resourceSchema = new mongoose.Schema({
title: { type: String, required: true, trim: true },
description: { type: String, trim: true, default: '' },
subject: { type: String, trim: true, default: 'Physics' },
classGrade: { type: String, trim: true, default: 'All Classes' },
chapter: { type: String, trim: true, default: '' },
pdf: {
data: { type: Buffer, required: true },
contentType: { type: String, default: 'application/pdf' },
fileName: { type: String, required: true },
fileSize: { type: Number, default: 0 }
},
uploadedBy: { type: String, default: 'JD Sir (Admin)' },
createdAt: { type: Date, default: Date.now }
});
```

## C. Student Router: Registration & Login (Backend/Route/studentRouter.js)

Demonstrates the password security lifecycle. Notice the use of bcrypt.hash(password, 10) for registration and bcrypt.compare(password, student.password) during login.

```
// 1. Password Hashing & Registration
const hashedPassword = await bcrypt.hash(password, 10);
const newStudent = new Student({
fullname, email: normalizedEmail, password: hashedPassword, phone, classGrade
});
await newStudent.save();

// 2. Authentication & JWT Token Issuance
const isMatch = await bcrypt.compare(password, student.password);
if (!isMatch) return res.status(401).json({ success: false, message: 'Invalid credentials' });

const token = jwt.sign(
{ id: student._id, email: student.email, role: 'student' },
process.env.JWT_KEY, { expiresIn: '7d' }
);
```

## D. Resource Router: Projection Optimization & Streaming (Backend/Route/resourceRouter.js)

Key architectural highlight: .select('-pdf.data') strips out multi-megabyte binary buffers from the metadata query, ensuring instant response times. The inline and download streaming routes set proper HTTP Content-Disposition headers.

```
// 1. Lightweight Query with Field Projection
const resources = await Resource.find(query).select('-pdf.data').sort({ createdAt: -1 });

// 2. Inline PDF Viewing in Browser Tab
router.get('/:id/view', async (req, res) => {
const resource = await Resource.findById(req.params.id);
res.setHeader('Content-Type', 'application/pdf');
res.setHeader('Content-Disposition', `inline; filename="${resource.pdf.fileName}"`);
res.send(resource.pdf.data);
});
```

# 4. Deep-Dive Frontend Code Walkthrough

## A. Dynamic Base URL Resolver (Frontend/src/utils/api.js)

Eliminates hardcoded localhost URLs. If executed in a local browser (hostname === 'localhost'), it connects to the local backend port 3000; in deployed environments, it automatically defaults to Render production.

## B. Global Reactive Auth State in Navbar (Frontend/src/components/Navbar.jsx)

Demonstrates an elegant event-driven state update without adding Redux boilerplate. Dispatches a custom DOM event studentAuthChange whenever a student logs in or logs out, causing the navigation bar to immediately sync its greeting badge.

```
useEffect(() => {
const loadStudent = () => {
const saved = localStorage.getItem('studentUser');
setStudent(saved ? JSON.parse(saved) : null);
};
loadStudent();
window.addEventListener('studentAuthChange', loadStudent);
return () => window.removeEventListener('studentAuthChange', loadStudent);
}, []);
```

## C. Study Materials Page (Frontend/src/components/Resources.jsx)

Features multi-criteria search and filter chips (Mechanics, Electromagnetism, Optics, Class 11, Class 12). Calculates human-readable file sizes dynamically (KB/MB) and renders cards with inline view and download triggers.

# 5. Top Technical Interview Questions & Answers

### Q1: Walk me through the architecture and how a request travels from the React frontend to MongoDB.

Our project follows a 3-tier MERN client-server architecture: (1) Presentation layer in React 19 SPA running in the browser, (2) Application/API layer in Node.js & Express 5, and (3) Data layer in MongoDB Atlas managed through Mongoose 8. When a user triggers an action (e.g. logging in), React Hook Form validates the input client-side. Axios sends an asynchronous HTTP POST request with credentials. Express passes the request through CORS and express.json() middlewares. The studentRouter executes business logic, queries MongoDB via Mongoose, compares password hashes with bcrypt, and returns a signed JWT token. The client stores the user session in localStorage and dispatches a studentAuthChange event to immediately update the navigation bar.

### Q2: Why use Bcrypt instead of SHA-256 for password hashing?

SHA-256 is designed to be fast for file verification and cryptographic proofs. Because it is fast, modern GPUs can compute billions of SHA-256 hashes per second, making offline rainbow table and brute-force dictionary attacks trivial. In contrast, Bcrypt is deliberately slow and adaptive. It incorporates a built-in salt to defeat rainbow tables and has a configurable work factor (salt rounds = 10, meaning 2^10 iterations). This deliberate computational overhead makes brute-force attacks computationally prohibitive even if the database is compromised.

### Q3: How do you handle file uploads, and what are the trade-offs of storing file buffers directly in MongoDB?

We use Multer with memoryStorage(), which intercepts multipart/form-data and buffers the incoming file into memory as req.file.buffer. We then store the binary buffer directly in a Mongoose document. The advantage for this institute portal is a self-contained, zero-dependency architecture without requiring external cloud accounts (AWS/Cloudinary), with atomic deletions (deleting a resource record purges the file). The trade-off is MongoDB's 16MB document size limit and database RAM utilization. In a massive enterprise system, we would upload the buffer to AWS S3 and persist only the resulting CDN URL in MongoDB.

### Q4: How did you optimize database queries when listing resources containing large PDFs?

By default, Mongoose loads every field including pdf.data. Pulling 20 PDF documents of 5MB each would transfer 100MB of binary data over the wire on every single page view! We resolved this using Mongoose Field Projection: Resource.find(query).select('-pdf.data'). This reduces the payload from 100MB down to ~2KB of lightweight JSON metadata (title, subject, file size, date). The binary data is only loaded on demand when a student explicitly clicks on /api/resources/:id/view or /api/resources/:id/download.

### Q5: How does the server differentiate between opening a PDF in the browser vs forcing a download?

This is controlled using HTTP response headers: For inline viewing, we set Content-Disposition: inline; filename='notes.pdf', which tells the browser's built-in PDF viewer to open and render the document inside a new browser tab. For forced downloading, we set Content-Disposition: attachment; filename='notes.pdf' and Content-Type: application/octet-stream, which instructs the browser to open the OS Save As prompt.

### Q6: How would you scale this application from 500 students to 100,000 active students?

I would implement a 4-step scaling strategy: (1) Offload file buffers from MongoDB to Amazon S3 edge-cached through AWS CloudFront CDN. (2) Deploy a Redis caching layer in front of Express to cache frequent read endpoints (/api/resources, /images), cutting database queries by 80%+. (3) Add compound database indexes ({ subject: 1, classGrade: 1, createdAt: -1 }) and MongoDB Atlas read replicas. (4) Horizontally scale the stateless Express server across multiple Docker containers behind an Nginx or AWS Application Load Balancer (ALB).